

Lecture 11: Time and Space Complexity Analysis

1. The Core Paradigm of Complexity

A common misconception among beginners is that time complexity measures the exact execution time of a program in seconds. This is mathematically incorrect.

- **Hardware Independence:** The exact run-time of an algorithm depends heavily on the computer architecture, background OS processes, hardware speed, and memory allocation. We need a mathematical metric independent of hardware.
- **Time Complexity:** The theoretical evaluation of how the runtime of an algorithm *grows* strictly as a mathematical function of the length of the input data (N).
- **Space Complexity:** The theoretical evaluation of the total auxiliary (extra) memory space required by an algorithm to execute, relative to the input size (N).

2. Asymptotic Notations

In algorithm analysis, we evaluate the boundaries of our function's growth using Asymptotic Notations.

- **Big-O Notation (O):** Represents the strict **Upper Bound** (Worst-Case Scenario). It mathematically guarantees that the algorithm will never exceed this time limit. This is the primary metric used in competitive programming.
- **Theta Notation (Θ):** Represents the **Average Bound**. It dictates the exact asymptotic behavior of the algorithm in a typical scenario.
- **Omega Notation (Ω):** Represents the **Lower Bound** (Best-Case Scenario). It shows the absolute minimum time required if the input is perfectly optimal.

3. Complexity Hierarchy & Growth Rates

Understanding which algorithm out-performs another requires memorizing the strict hierarchy of mathematical growth.

Complexity	Notation	Description	Example Algorithm
Constant	$O(1)$	Execution time is fixed, regardless of N .	Array index lookup, even/odd bitwise check.
Logarithmic	$O(\log N)$	Data size is divided by a fraction on each step.	Binary Search.

Complexity	Notation	Description	Example Algorithm
Linear	$O(N)$	Time scales perfectly 1:1 with input size.	Linear Search, Array traversal.
Linearithmic	$O(N \log N)$	Linear loops enclosing logarithmic subdivisions.	Merge Sort, Quick Sort (average).
Quadratic	$O(N^2)$	Time scales quadratically. Nested loops.	Bubble Sort, checking all pairs.
Cubic	$O(N^3)$	Three nested loops processing N .	Matrix multiplication (naive).
Exponential	$O(2^N)$	Time doubles with each new element added.	Recursive Fibonacci.
Factorial	$O(N!)$	Computes all mathematical permutations.	Traveling Salesperson brute force.

Strict Mathematical Order (Best to Worst):

$O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(N^3) < O(2^N) < O(N!)$

4. The 10^8 Operations Rule (Competitive Programming Hack)

Most modern CPU architectures can securely evaluate roughly 10^8 **operations per second**. When tackling LeetCode or CP problems with strict 1-second time limits, you must look at the given constraint for N and instantly deduce the maximum allowable time complexity.

Problem Constraint on Input Size (N)	Maximum Allowable Complexity	Target Algorithm Approach
$N \leq [10, 11]$	$O(N!), O(N^6), O(2^N \times N^2)$	Permutations, Backtracking
$N \leq [15, 18]$	$O(2^N \times N)$	Bitmask DP, Advanced Backtracking
$N \leq 100$	$O(N^4)$	4D Dynamic Programming
$N \leq 400$	$O(N^3)$	Floyd-Warshall, 3D DP
$N \leq 2000$	$O(N^2 \log N)$	Nested loops + Binary Search
$N \leq 10^4$	$O(N^2)$	Bubble Sort, Nested linear traversals
$N \leq 10^6$	$O(N \log N)$	Merge Sort, Sorting + Two Pointers
$N \leq 10^8$	$O(N)$ or $O(\log N)$	Strictly Single Pass Traversals, Binary Search

(If you write an $O(N^2)$ algorithm for a problem where $N \leq 10^5$, it will mathematically require $(10^5)^2 = 10^{10}$ operations. This shatters the 10^8 limit and triggers a Time Limit Exceeded (TLE) error).

Loop Deconstruction & Analysis Protocols

Case 1: Sequential Loops (Additive Complexity)

1. Problem Statement: Analyze the complexity of multiple loops executing sequentially (not nested).

C++

```
int a = 0;
// Loop 1
for(int i = 0; i < n; i++) {
    a++;
}
// Loop 2
for(int j = 0; j < m; j++) {
    a++;
}
```

2. Algorithmic Evaluation Steps:

1. Isolate the first loop. It strictly traverses from 0 to N . Its individual complexity is $O(N)$.
2. Isolate the second loop. It strictly traverses from 0 to M . Its individual complexity is $O(M)$.
3. Because the loops execute one after the other, their mathematical costs are summed together.
4. Total Complexity: $O(N + M)$.

(Note: If the second loop was also bound by N , the total would be $O(2N)$, which asymptotically drops the constant to become strictly $O(N)$).

Case 2: Nested Dependent Loops (Multiplicative Complexity)

1. Problem Statement: Analyze the worst-case time complexity of nested loops where the inner loop's boundary relies directly on the outer loop's iterator.

C++

```
int a = 0;
for(int i = 0; i < n; i++) {
    for(int j = n; j > i; j--) {
        a = a + i + j;
    }
}
```

```
}  
}
```

2. Algorithmic Evaluation Steps (Mathematical Derivation):

1. Track Outer Loop ($i = 0$): Inner loop j runs from N down to 1. It executes N **times**.
2. Track Outer Loop ($i = 1$): Inner loop j runs from N down to 2. It executes $N - 1$ **times**.
3. Track Outer Loop ($i = 2$): Inner loop j runs from N down to 3. It executes $N - 2$ **times**.
4. Track Outer Loop ($i = N - 1$): Inner loop j runs from N down to N . It executes 1 **time**.

3. Complexity Analysis Table:

Metric	Complexity	Justification
Total Operations	$\frac{N(N+1)}{2}$	The total executions form an arithmetic progression: $N + (N - 1) + (N - 2) \cdots + 1$. The mathematical sum of the first N natural numbers is exactly $\frac{N^2}{2} + \frac{N}{2}$.
Asymptotic Time Complexity	$O(N^2)$	When applying Big-O notation, we drop all non-dominant mathematical terms (like $\frac{N}{2}$) and scale constants (like $\frac{1}{2}$). We are strictly left with the dominant quadratic factor, N^2 .
Space Complexity	$O(1)$	Memory usage is entirely restricted to the scalar tracking variables <code>i</code> , <code>j</code> , and <code>a</code> .